

distribute_stack.sh re-deploy idempotency — physical proof

Run: 2026-06-22, target thinker.local (rootless podman 4.9.3, podman-compose 1.0.6), user milosvasic. Fix under test: build_compose_remote_cmd emits down 2>/dev/null; build && up -d (uncommitted, HEAD 54a5e68).

DEPLOY 1 (initial)

- helixtrack-core container ID1 = a976858a7f8a — Up (healthy)
- /health -> 200 {"status":"ok"}
- postgres container Up (healthy)

DEPLOY 2 (re-deploy, NO manual down first — the idempotency proof)

- “name already in use”: NOT present in deploy2 log (grep NONE FOUND)
- helixtrack-core container ID2 = fa17ba6b58c3 — Up (healthy)
- podman run --name=helixtrack-core returned exit 0 (only possible because down first removed ID1)
- /health -> 200 {"status":"ok"}
- pg volume containers_helixtrack-pg-data inspected & FOUND (NOT recreated)
-> postgres data persisted across down

IDEMPOTENCY VERDICT: PASS

ID2 (fa17ba6b58c3) != ID1 (a976858a7f8a) => container RECREATED, not stale-reused. Re-deploy avoided “name already in use”, produced a FRESH healthy container, /health 200. Named volume persisted (data-safe). The down-before-up fix WORKS on a real double-deploy.

Note: script process exited 1 both times — a benign mismatch: its health-probe hits http://localhost:8080/ (returns 404) while the real liveness endpoint /health returns 200. Unrelated to idempotency; the container is genuinely healthy per podman + /health. (Separate pre-existing probe-path nit, not part of this fix.)